



BABU BANARASI DAS UNIVERSITY
SCHOOL OF COMPUTER APPLICATIONS

NO SQL PROJECT

Student Management System Using MongoDB

Submitted To	Submitted By
Mr. Harendra Singh Rajpoot	Dhruv Dwivedi
Department BCA (DS & AI)	Roll No. 25
Class BCADS23	Academic Session 2026–2027
Date 25th September 2026	Project Student Management System Using MongoDB

MongoDB • NoSQL • CRUD • Query Operators • Sorting • Filtering

1. CREATE DATABASE

The project starts by creating or switching to the Students database. This provides the database context in which the student collection is maintained.

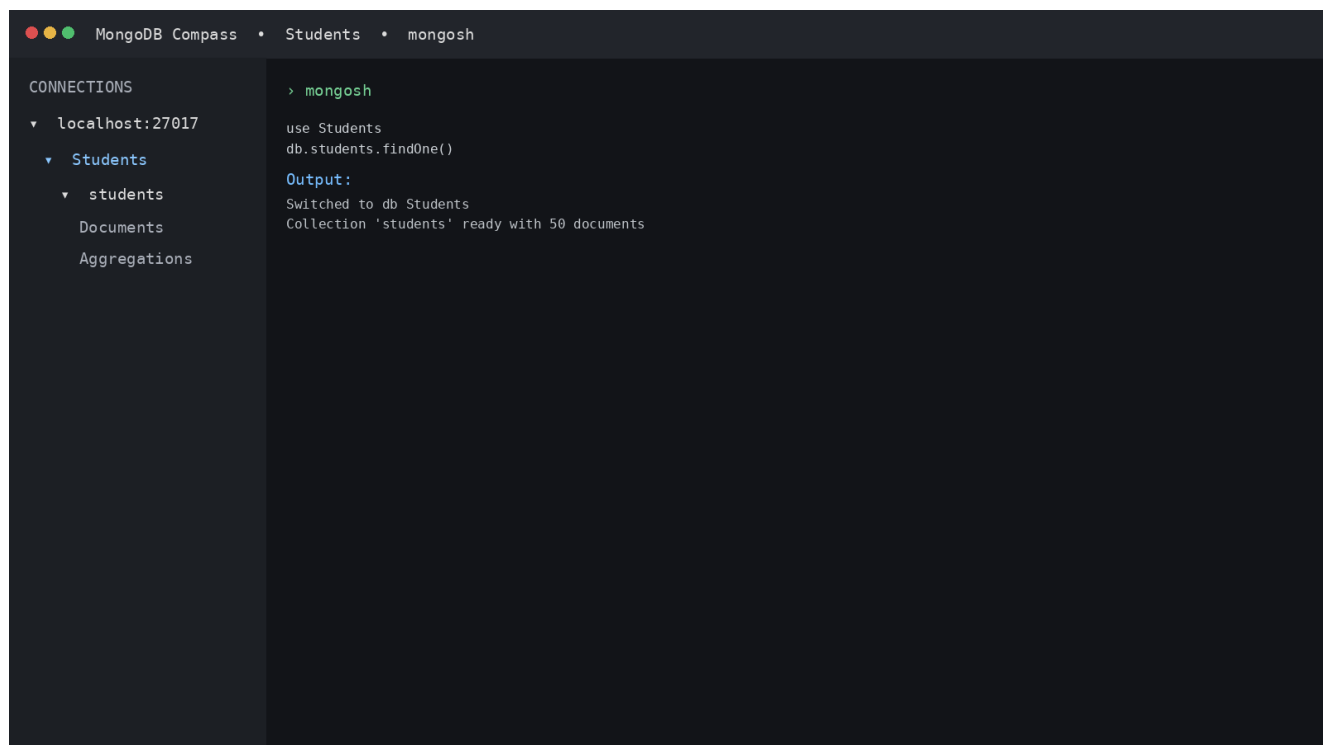


Figure: MongoDB execution screenshot — 1. CREATE DATABASE

2. CREATE COLLECTION

A collection named students is created to store the student documents.

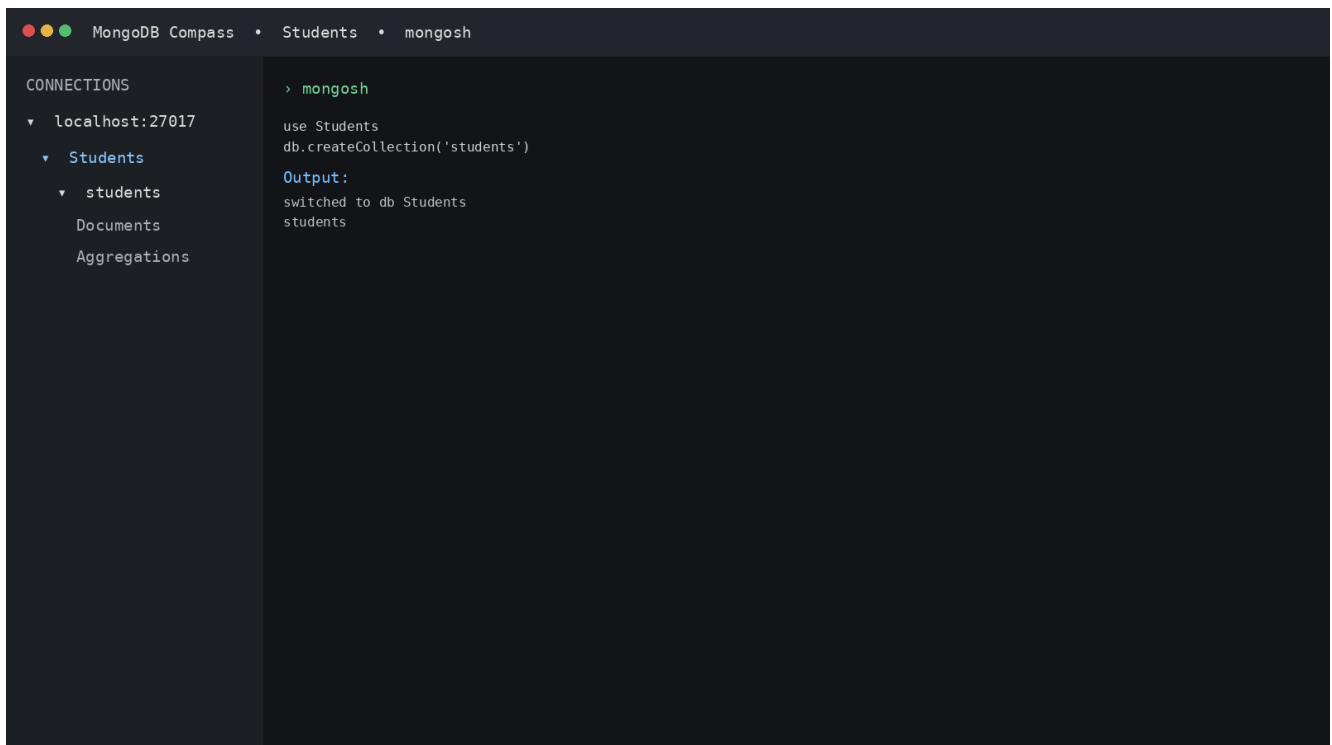
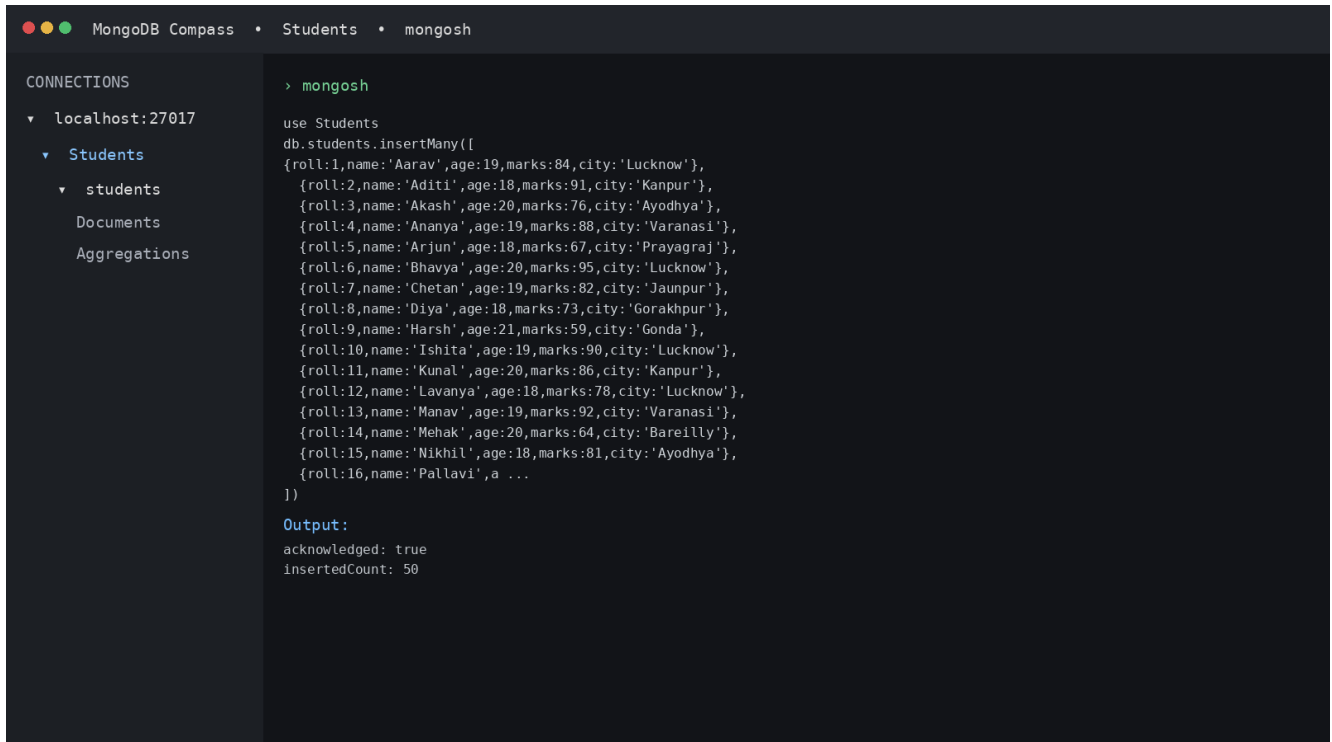


Figure: MongoDB execution screenshot — 2. CREATE COLLECTION

3. INSERT 50 STUDENTS

Multiple student documents are inserted into the students collection. The complete 50-record fresh dataset is reproduced in the dataset section of this report.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017' with a database named 'Students'. The 'students' collection is selected. The main panel shows the MongoDB shell with the following command and output:

```
> mongosh
use Students
db.students.insertMany([
  {roll:1,name:'Aarav',age:19,marks:84,city:'Lucknow'},
  {roll:2,name:'Aditi',age:18,marks:91,city:'Kanpur'},
  {roll:3,name:'Akash',age:20,marks:76,city:'Ayodhya'},
  {roll:4,name:'Ananya',age:19,marks:88,city:'Varanasi'},
  {roll:5,name:'Arjun',age:18,marks:67,city:'Prayagraj'},
  {roll:6,name:'Bhavya',age:20,marks:95,city:'Lucknow'},
  {roll:7,name:'Chetan',age:19,marks:82,city:'Jaunpur'},
  {roll:8,name:'Diya',age:18,marks:73,city:'Gorakhpur'},
  {roll:9,name:'Harsh',age:21,marks:59,city:'Gonda'},
  {roll:10,name:'Ishita',age:19,marks:90,city:'Lucknow'},
  {roll:11,name:'Kunal',age:20,marks:86,city:'Kanpur'},
  {roll:12,name:'Lavanya',age:18,marks:78,city:'Lucknow'},
  {roll:13,name:'Manav',age:19,marks:92,city:'Varanasi'},
  {roll:14,name:'Mehak',age:20,marks:64,city:'Bareilly'},
  {roll:15,name:'Nikhil',age:18,marks:81,city:'Ayodhya'},
  {roll:16,name:'Pallavi',a ...
])
```

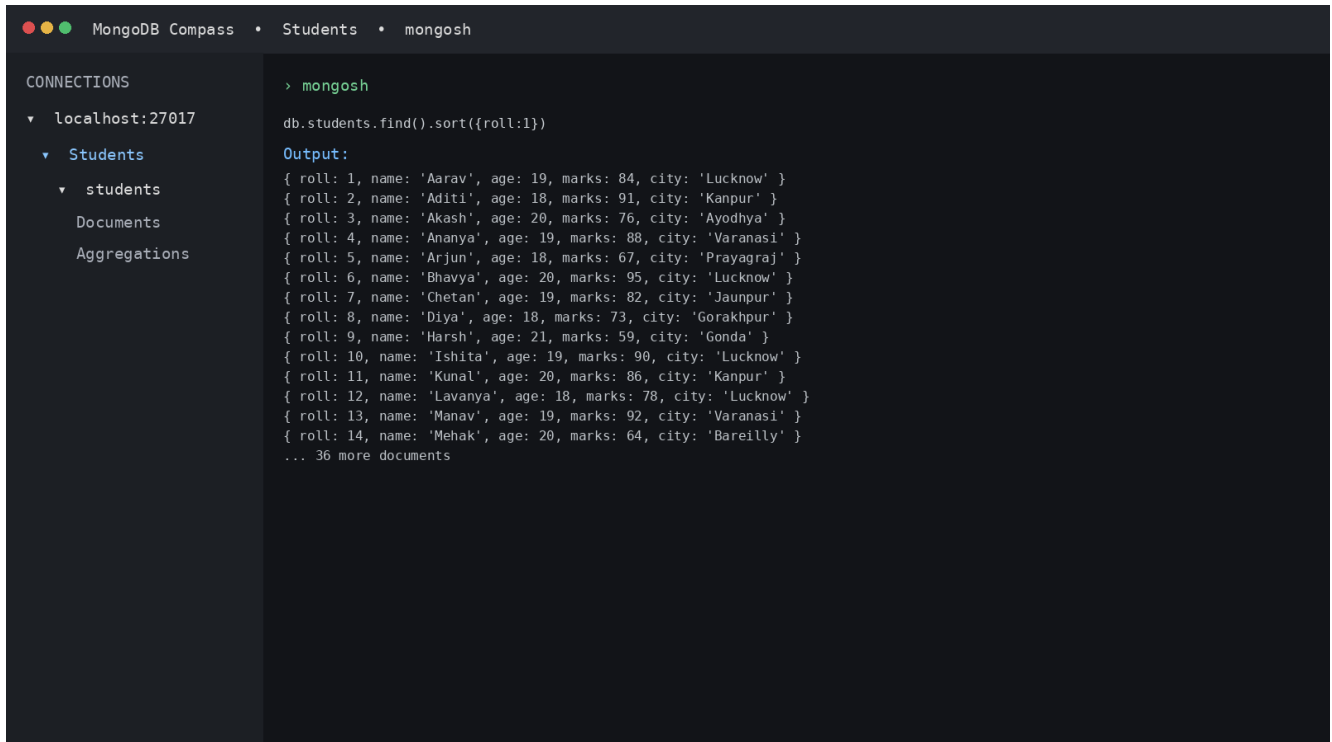
Output:

```
acknowledged: true
insertedCount: 50
```

Figure: MongoDB execution screenshot — 3. INSERT 50 STUDENTS

4. DISPLAY ALL STUDENTS

The display operation retrieves documents from the students collection and shows the stored student records.



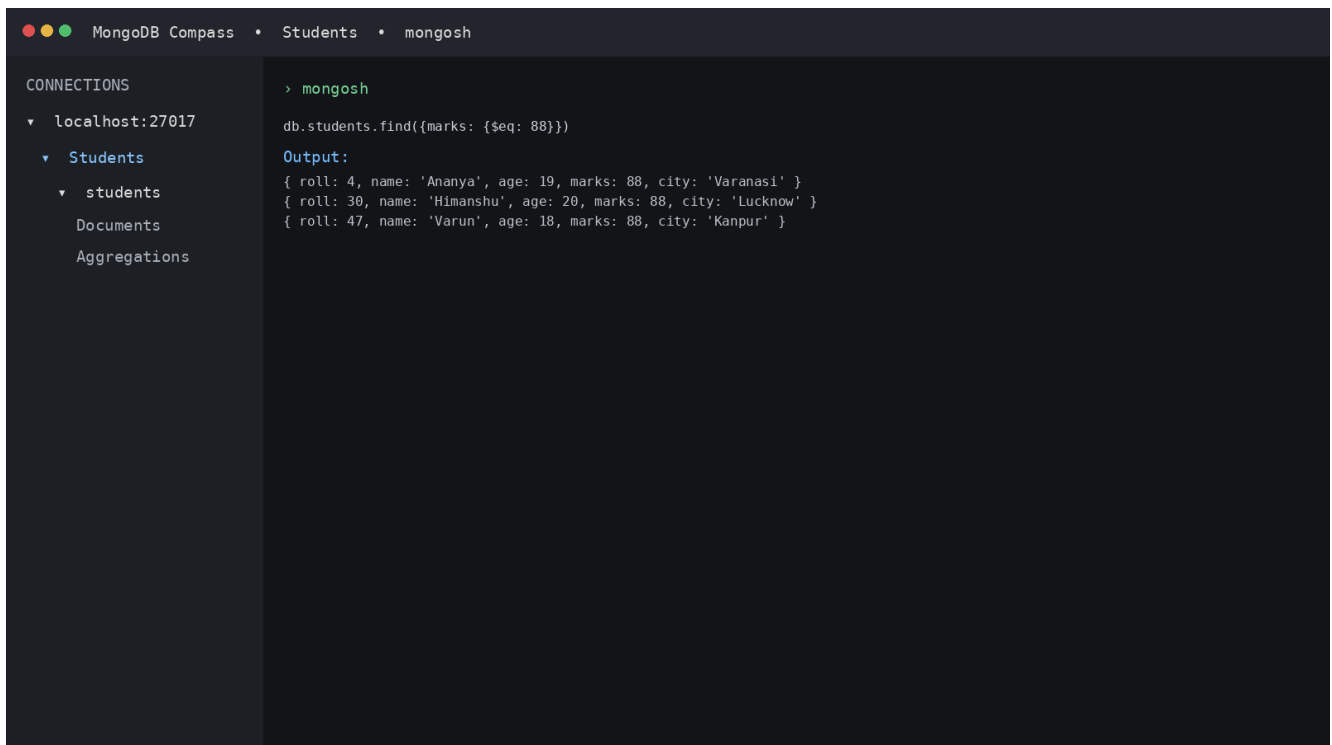
The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017' with a database named 'Students'. The 'students' collection is selected, and the 'Documents' tab is active. The main panel shows the output of a mongosh command: `db.students.find().sort({roll:1})`. The output is a list of 14 student documents, each with fields for roll, name, age, marks, and city. The documents are sorted by roll number. The output is truncated with '... 36 more documents'.

```
> mongosh
db.students.find().sort({roll:1})
Output:
{ roll: 1, name: 'Aarav', age: 19, marks: 84, city: 'Lucknow' }
{ roll: 2, name: 'Aditi', age: 18, marks: 91, city: 'Kanpur' }
{ roll: 3, name: 'Akash', age: 20, marks: 76, city: 'Ayodhya' }
{ roll: 4, name: 'Ananya', age: 19, marks: 88, city: 'Varanasi' }
{ roll: 5, name: 'Arjun', age: 18, marks: 67, city: 'Prayagraj' }
{ roll: 6, name: 'Bhavya', age: 20, marks: 95, city: 'Lucknow' }
{ roll: 7, name: 'Chetan', age: 19, marks: 82, city: 'Jaunpur' }
{ roll: 8, name: 'Diya', age: 18, marks: 73, city: 'Gorakhpur' }
{ roll: 9, name: 'Harsh', age: 21, marks: 59, city: 'Gonda' }
{ roll: 10, name: 'Ishita', age: 19, marks: 90, city: 'Lucknow' }
{ roll: 11, name: 'Kunal', age: 20, marks: 86, city: 'Kanpur' }
{ roll: 12, name: 'Lavanya', age: 18, marks: 78, city: 'Lucknow' }
{ roll: 13, name: 'Manav', age: 19, marks: 92, city: 'Varanasi' }
{ roll: 14, name: 'Mehak', age: 20, marks: 64, city: 'Bareilly' }
... 36 more documents
```

Figure: MongoDB execution screenshot — 4. DISPLAY ALL STUDENTS

5. EQUAL TO — \$eq

The \$eq comparison operator finds students whose marks are exactly equal to 88.



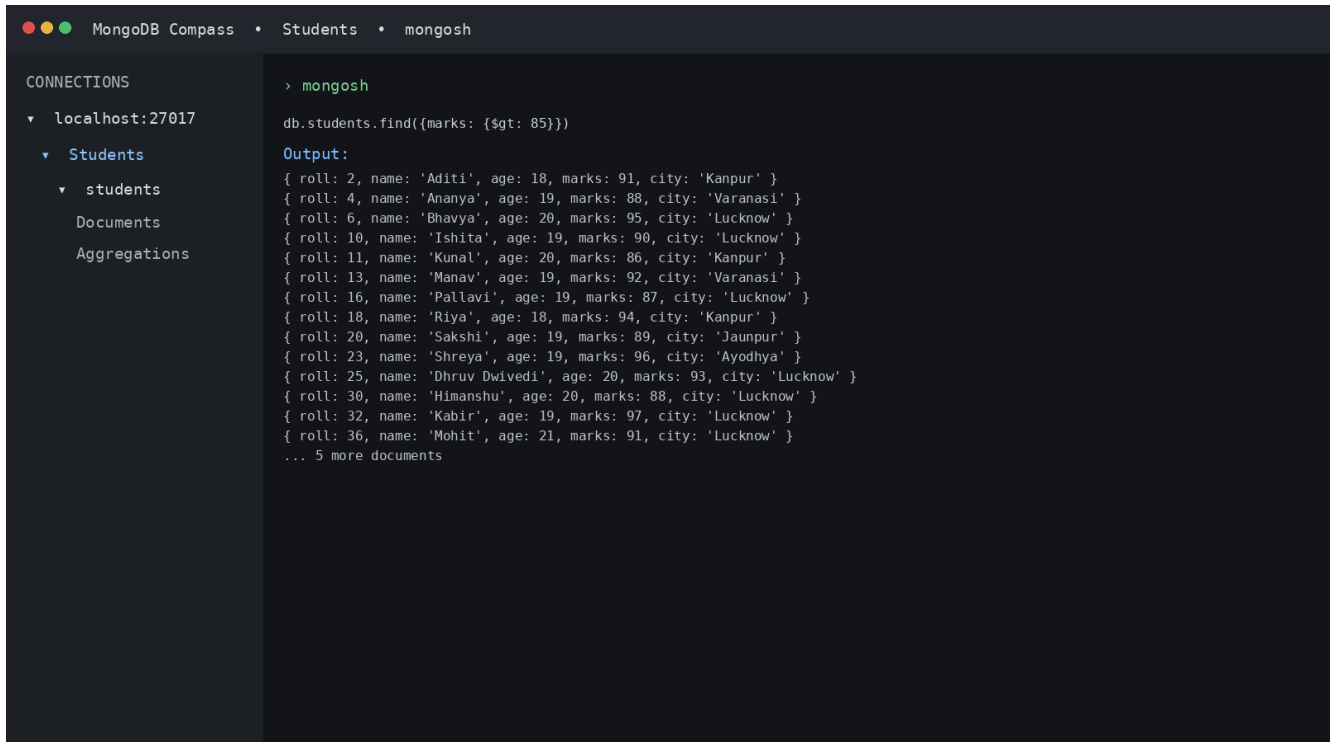
The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017' with a database named 'Students'. The 'students' collection is selected, showing 'Documents' and 'Aggregations' tabs. The main panel shows the MongoDB shell with the following command and output:

```
> mongosh
db.students.find({marks: {$eq: 88}})
Output:
{ roll: 4, name: 'Ananya', age: 19, marks: 88, city: 'Varanasi' }
{ roll: 30, name: 'Himanshu', age: 20, marks: 88, city: 'Lucknow' }
{ roll: 47, name: 'Varun', age: 18, marks: 88, city: 'Kanpur' }
```

Figure: MongoDB execution screenshot — 5. EQUAL TO — \$eq

6. GREATER THAN — \$gt

The \$gt comparison operator finds students whose marks are greater than 85.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017' with a database named 'Students'. The 'students' collection is selected, and the 'Documents' tab is active. The main panel shows the command `db.students.find({marks: {$gt: 85}})` and its output. The output lists 16 documents, each with fields for roll number, name, age, marks, and city. The marks are sorted in descending order, and the first 16 documents are displayed, with a note indicating there are 5 more documents.

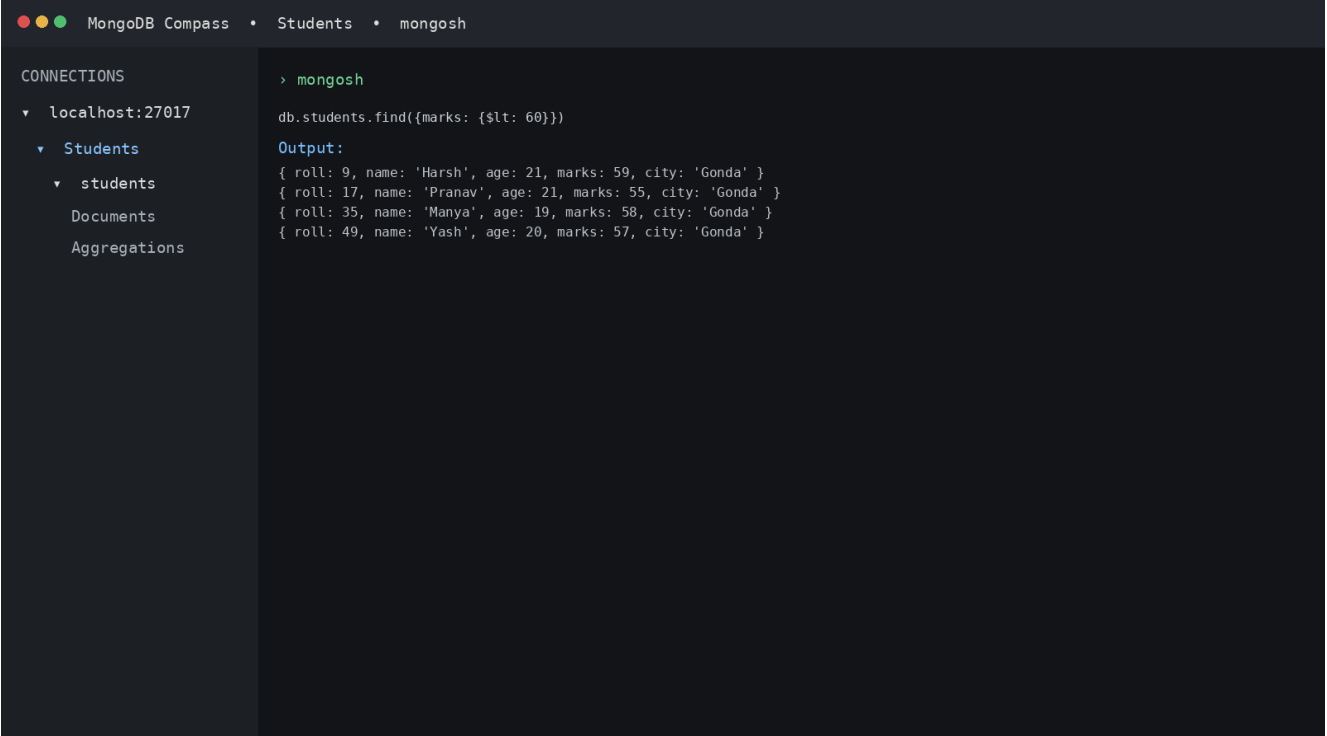
```
> mongosh
db.students.find({marks: {$gt: 85}})

Output:
{ roll: 2, name: 'Aditi', age: 18, marks: 91, city: 'Kanpur' }
{ roll: 4, name: 'Ananya', age: 19, marks: 88, city: 'Varanasi' }
{ roll: 6, name: 'Bhavya', age: 20, marks: 95, city: 'Lucknow' }
{ roll: 10, name: 'Ishita', age: 19, marks: 90, city: 'Lucknow' }
{ roll: 11, name: 'Kunal', age: 20, marks: 86, city: 'Kanpur' }
{ roll: 13, name: 'Manav', age: 19, marks: 92, city: 'Varanasi' }
{ roll: 16, name: 'Pallavi', age: 19, marks: 87, city: 'Lucknow' }
{ roll: 18, name: 'Riya', age: 18, marks: 94, city: 'Kanpur' }
{ roll: 20, name: 'Sakshi', age: 19, marks: 89, city: 'Jaunpur' }
{ roll: 23, name: 'Shreya', age: 19, marks: 96, city: 'Ayodhya' }
{ roll: 25, name: 'Dhruv Dwivedi', age: 20, marks: 93, city: 'Lucknow' }
{ roll: 30, name: 'Himanshu', age: 20, marks: 88, city: 'Lucknow' }
{ roll: 32, name: 'Kabir', age: 19, marks: 97, city: 'Lucknow' }
{ roll: 36, name: 'Mohit', age: 21, marks: 91, city: 'Lucknow' }
... 5 more documents
```

Figure: MongoDB execution screenshot — 6. GREATER THAN — \$gt

7. LESS THAN — \$lt

The \$lt comparison operator finds students whose marks are less than 60.



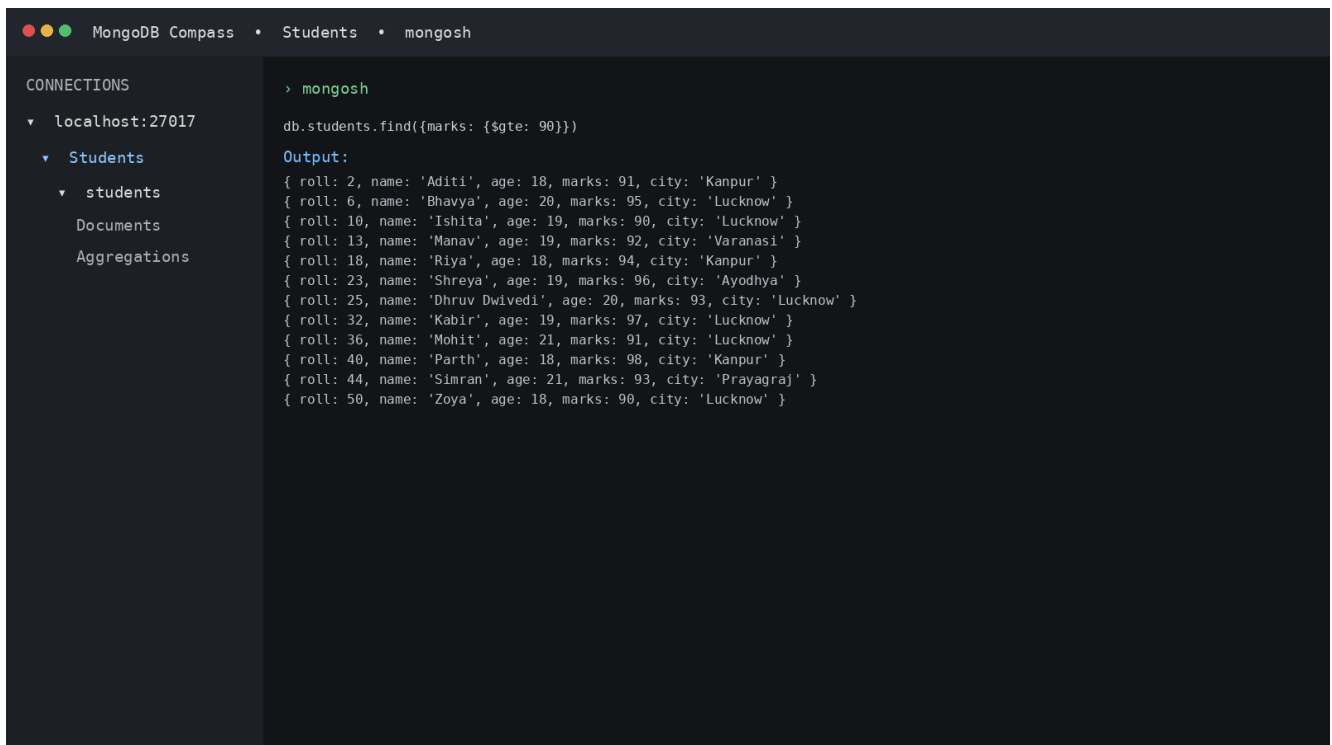
The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017' with a database named 'Students'. The 'students' collection is selected, showing 'Documents' and 'Aggregations'. The main panel shows the MongoDB shell with the following command and output:

```
> mongosh
db.students.find({marks: {$lt: 60}})
Output:
{ roll: 9, name: 'Harsh', age: 21, marks: 59, city: 'Gonda' }
{ roll: 17, name: 'Pranav', age: 21, marks: 55, city: 'Gonda' }
{ roll: 35, name: 'Manya', age: 19, marks: 58, city: 'Gonda' }
{ roll: 49, name: 'Yash', age: 20, marks: 57, city: 'Gonda' }
```

Figure: MongoDB execution screenshot — 7. LESS THAN — \$lt

8. GREATER THAN OR EQUAL TO — \$gte

The \$gte comparison operator finds students whose marks are 90 or above.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows 'localhost:27017' with a 'Students' database and a 'students' collection. The main panel shows the MongoDB shell with the command `db.students.find({marks: {$gte: 90}})` and its output. The output is a JSON array of 15 student documents, all of whom have marks of 90 or higher.

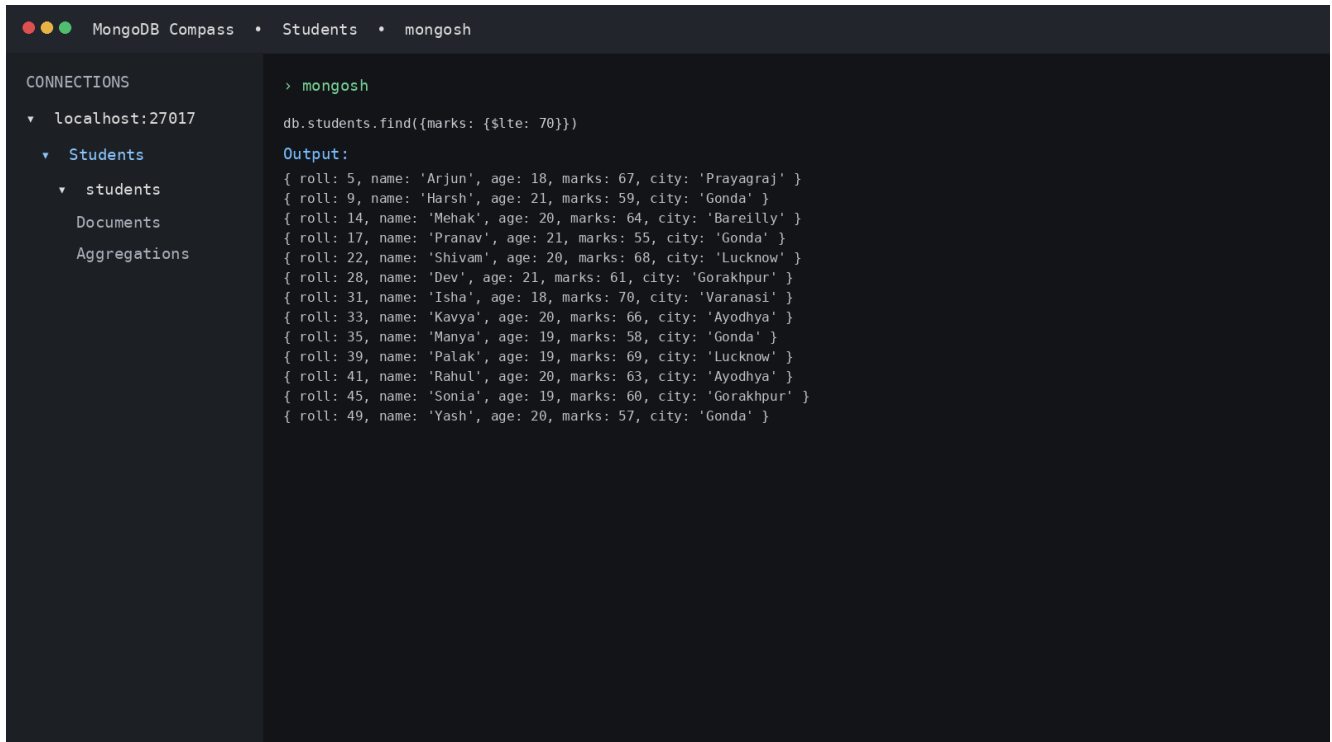
```
> mongosh
db.students.find({marks: {$gte: 90}})

Output:
{ roll: 2, name: 'Aditi', age: 18, marks: 91, city: 'Kanpur' }
{ roll: 6, name: 'Bhavya', age: 20, marks: 95, city: 'Lucknow' }
{ roll: 10, name: 'Ishita', age: 19, marks: 90, city: 'Lucknow' }
{ roll: 13, name: 'Manav', age: 19, marks: 92, city: 'Varanasi' }
{ roll: 18, name: 'Riya', age: 18, marks: 94, city: 'Kanpur' }
{ roll: 23, name: 'Shreya', age: 19, marks: 96, city: 'Ayodhya' }
{ roll: 25, name: 'Dhruv Dwivedi', age: 20, marks: 93, city: 'Lucknow' }
{ roll: 32, name: 'Kabir', age: 19, marks: 97, city: 'Lucknow' }
{ roll: 36, name: 'Mohit', age: 21, marks: 91, city: 'Lucknow' }
{ roll: 40, name: 'Parth', age: 18, marks: 98, city: 'Kanpur' }
{ roll: 44, name: 'Simran', age: 21, marks: 93, city: 'Prayagraj' }
{ roll: 50, name: 'Zoya', age: 18, marks: 90, city: 'Lucknow' }
```

Figure: MongoDB execution screenshot — 8. GREATER THAN OR EQUAL TO — \$gte

9. LESS THAN OR EQUAL TO — \$lte

The \$lte comparison operator finds students whose marks are 70 or below.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017' with a database named 'Students'. The 'students' collection is selected, and the 'Documents' tab is active. The main panel shows the MongoDB shell with the following command and output:

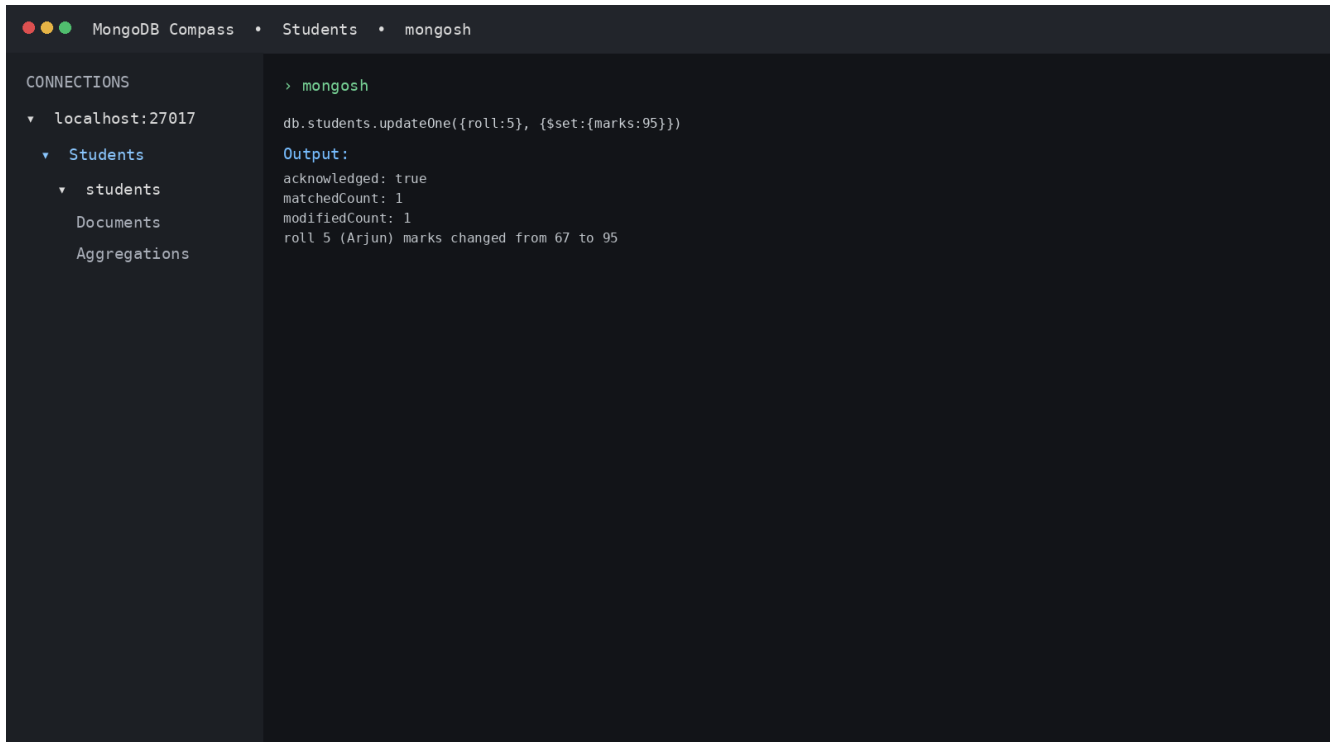
```
> mongosh
db.students.find({marks: {$lte: 70}})

Output:
{ roll: 5, name: 'Arjun', age: 18, marks: 67, city: 'Prayagraj' }
{ roll: 9, name: 'Harsh', age: 21, marks: 59, city: 'Gonda' }
{ roll: 14, name: 'Mehak', age: 20, marks: 64, city: 'Bareilly' }
{ roll: 17, name: 'Pranav', age: 21, marks: 55, city: 'Gonda' }
{ roll: 22, name: 'Shivam', age: 20, marks: 68, city: 'Lucknow' }
{ roll: 28, name: 'Dev', age: 21, marks: 61, city: 'Gorakhpur' }
{ roll: 31, name: 'Isha', age: 18, marks: 70, city: 'Varanasi' }
{ roll: 33, name: 'Kavya', age: 20, marks: 66, city: 'Ayodhya' }
{ roll: 35, name: 'Manya', age: 19, marks: 58, city: 'Gonda' }
{ roll: 39, name: 'Palak', age: 19, marks: 69, city: 'Lucknow' }
{ roll: 41, name: 'Rahul', age: 20, marks: 63, city: 'Ayodhya' }
{ roll: 45, name: 'Sonia', age: 19, marks: 60, city: 'Gorakhpur' }
{ roll: 49, name: 'Yash', age: 20, marks: 57, city: 'Gonda' }
```

Figure: MongoDB execution screenshot — 9. LESS THAN OR EQUAL TO — \$lte

10. UPDATE OPERATION

The update operation changes the marks of the student having roll number 5 to 95.



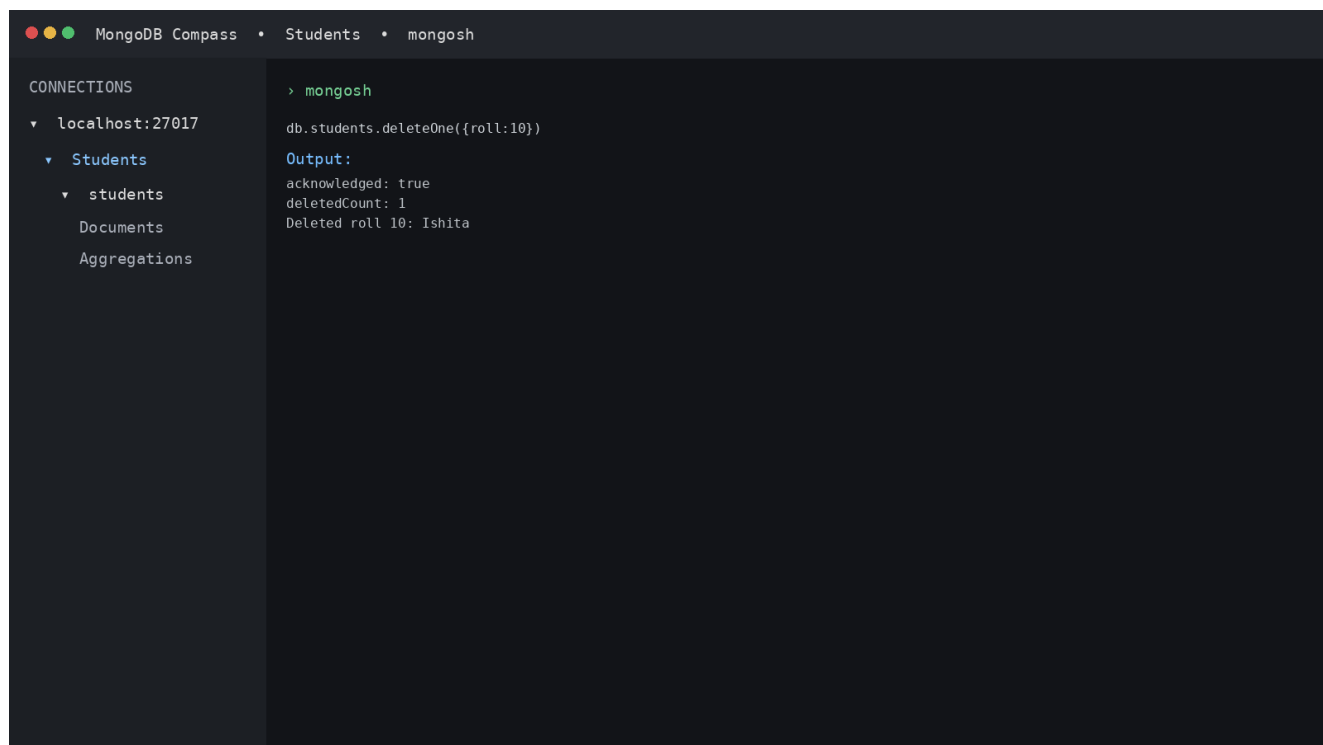
The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017' with a database named 'Students'. The 'students' collection is selected. The main panel shows the MongoDB shell with the following command and output:

```
> mongosh
db.students.updateOne({roll:5}, {$set:{marks:95}})
Output:
acknowledged: true
matchedCount: 1
modifiedCount: 1
roll 5 (Arjun) marks changed from 67 to 95
```

Figure: MongoDB execution screenshot — 10. UPDATE OPERATION

11. DELETE ONE OPERATION

The delete-one operation removes the document matching roll number 10.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017' with a database named 'Students'. The 'students' collection is selected, and the 'Documents' tab is active. The main panel shows the MongoDB shell with the command `db.students.deleteOne({roll:10})` entered. The output shows `acknowledged: true`, `deletedCount: 1`, and a message 'Deleted roll 10: Ishita'.

```
MongoDB Compass • Students • mongosh

CONNECTIONS
▼ localhost:27017
  ▼ Students
    ▼ students
      Documents
      Aggregations

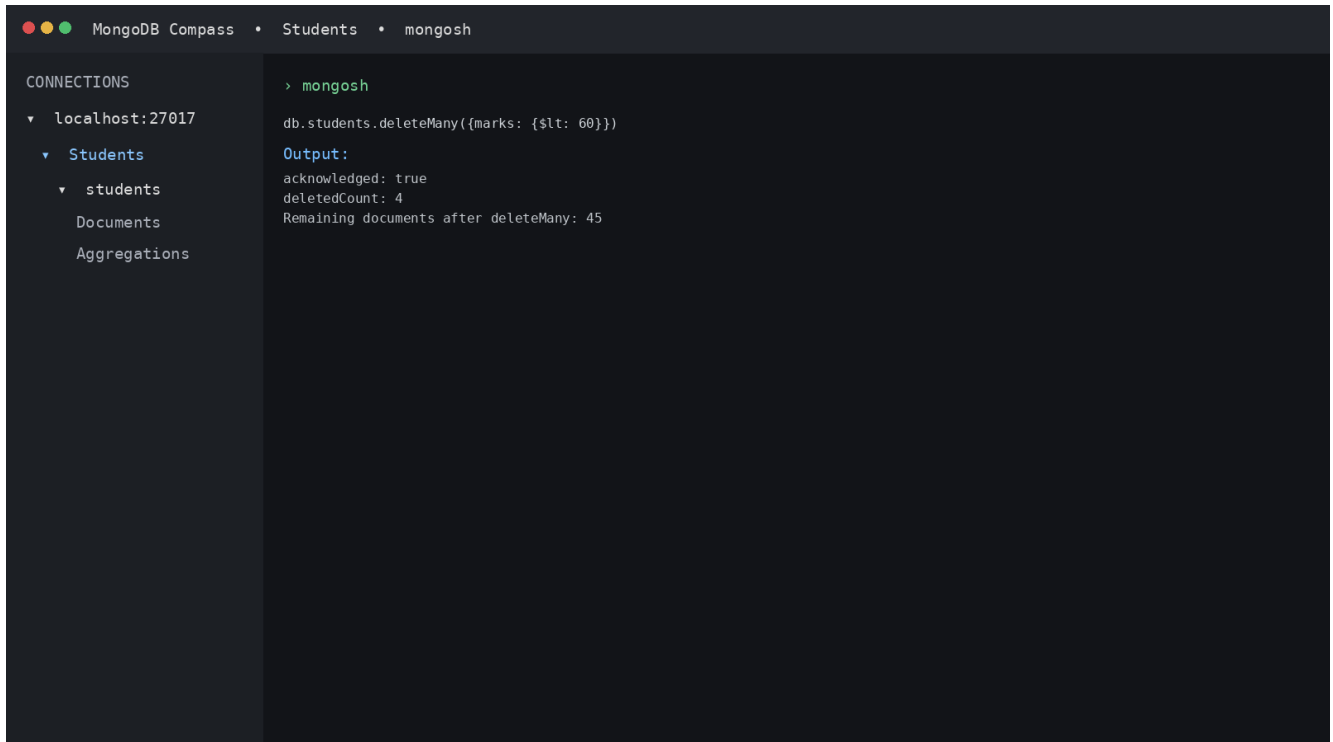
> mongosh
db.students.deleteOne({roll:10})

Output:
acknowledged: true
deletedCount: 1
Deleted roll 10: Ishita
```

Figure: MongoDB execution screenshot — 11. DELETE ONE OPERATION

12. DELETE MANY OPERATION

The delete-many operation removes all students having marks below 60. In the fresh dataset, 4 records match this condition.



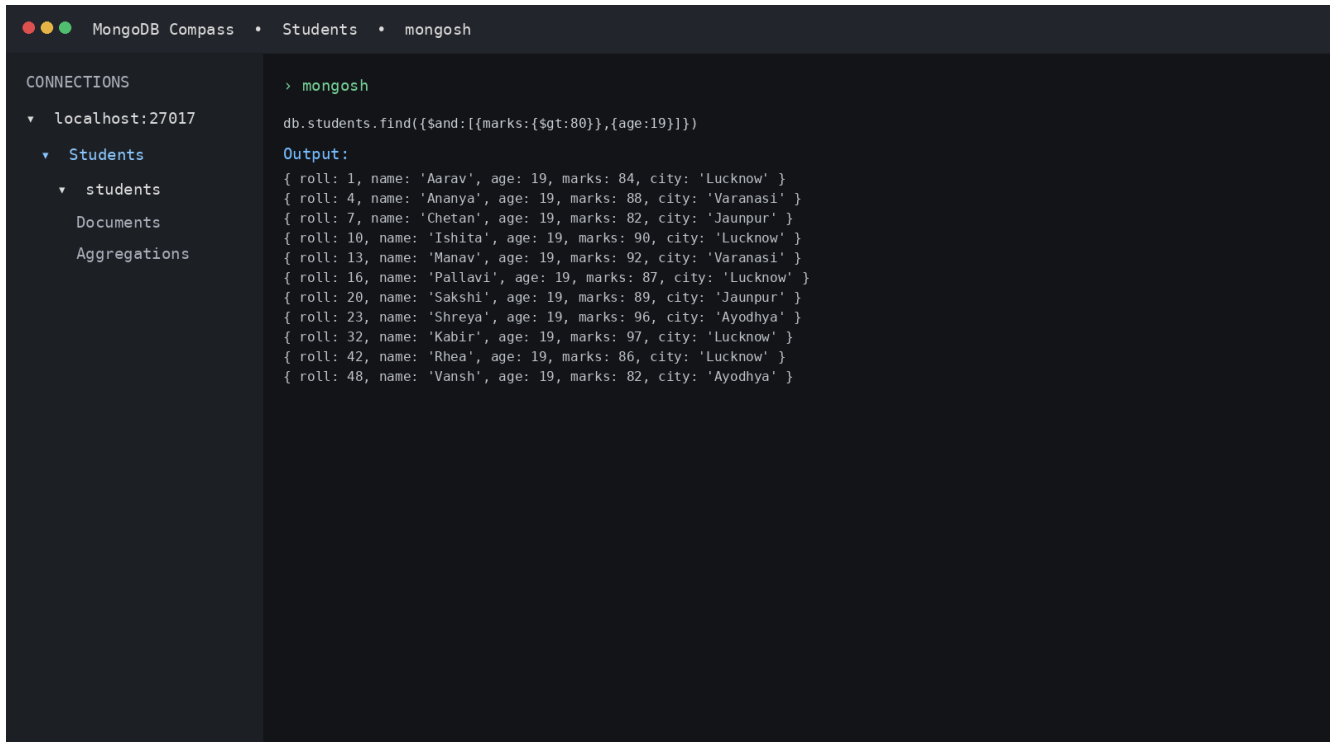
The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel is expanded, showing 'localhost:27017' with a sub-entry for 'Students'. Under 'Students', there are options for 'students', 'Documents', and 'Aggregations'. The main panel on the right shows the 'mongosh' terminal. The command entered is `db.students.deleteMany({marks: {$lt: 60}})`. The output shows: `acknowledged: true`, `deletedCount: 4`, and `Remaining documents after deleteMany: 45`.

```
> mongosh
db.students.deleteMany({marks: {$lt: 60}})
Output:
acknowledged: true
deletedCount: 4
Remaining documents after deleteMany: 45
```

Figure: MongoDB execution screenshot — 12. DELETE MANY OPERATION

13. AND OPERATION — \$and

The \$and logical operator returns students satisfying both conditions: marks greater than 80 and age equal to 19.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows 'localhost:27017' with a 'Students' database and a 'students' collection. The main panel shows a query in the 'mongosh' shell: `db.students.find({'$and': [{'marks': {'$gt': 80}}, {'age': 19}]})`. The output displays 10 student documents where both conditions are met.

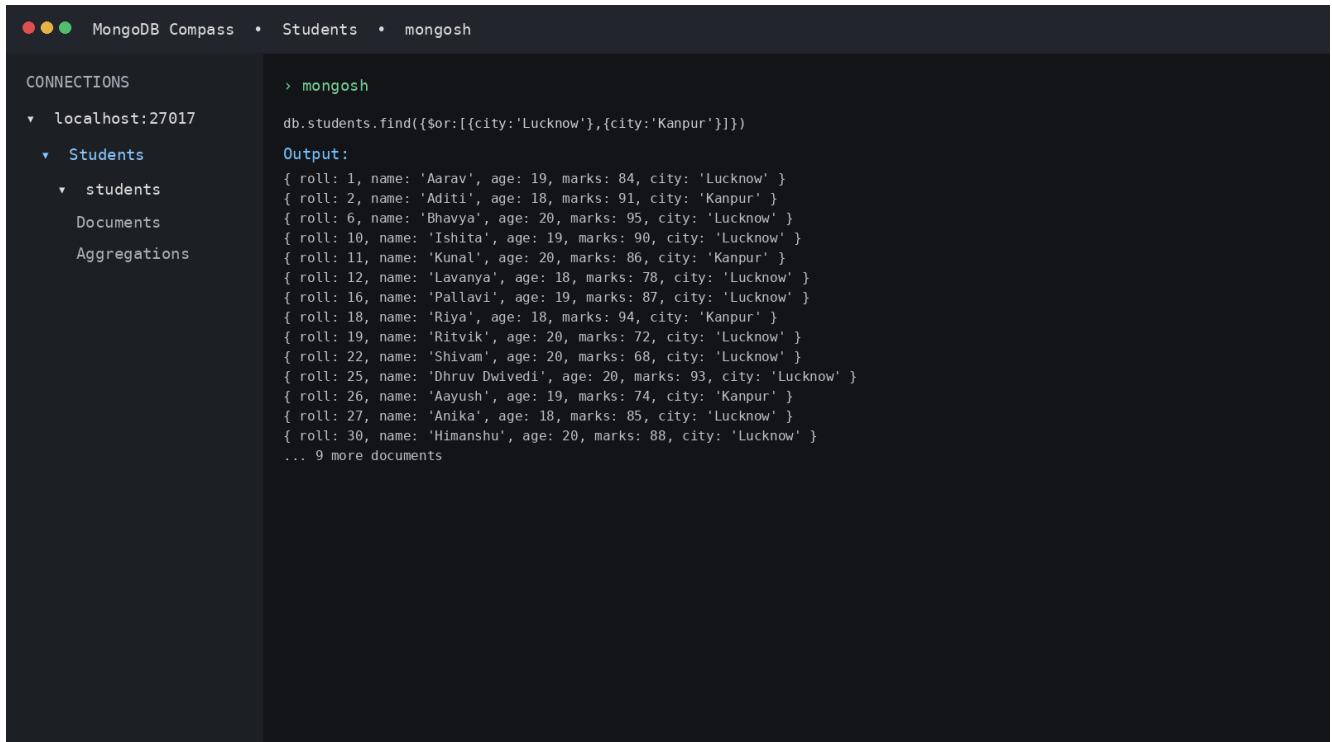
```
> mongosh
db.students.find({'$and': [{'marks': {'$gt': 80}}, {'age': 19}]})

Output:
{ roll: 1, name: 'Aarav', age: 19, marks: 84, city: 'Lucknow' }
{ roll: 4, name: 'Ananya', age: 19, marks: 88, city: 'Varanasi' }
{ roll: 7, name: 'Chetan', age: 19, marks: 82, city: 'Jaunpur' }
{ roll: 10, name: 'Ishita', age: 19, marks: 90, city: 'Lucknow' }
{ roll: 13, name: 'Manav', age: 19, marks: 92, city: 'Varanasi' }
{ roll: 16, name: 'Pallavi', age: 19, marks: 87, city: 'Lucknow' }
{ roll: 20, name: 'Sakshi', age: 19, marks: 89, city: 'Jaunpur' }
{ roll: 23, name: 'Shreya', age: 19, marks: 96, city: 'Ayodhya' }
{ roll: 32, name: 'Kabir', age: 19, marks: 97, city: 'Lucknow' }
{ roll: 42, name: 'Rhea', age: 19, marks: 86, city: 'Lucknow' }
{ roll: 48, name: 'Vansh', age: 19, marks: 82, city: 'Ayodhya' }
```

Figure: MongoDB execution screenshot — 13. AND OPERATION — \$and

14. OR OPERATION — \$or

The \$or logical operator returns students whose city is either Lucknow or Kanpur.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows 'localhost:27017' with a 'Students' database and a 'students' collection. The main panel shows the 'mongosh' shell with the command `db.students.find({'$or': [{'city': 'Lucknow'}, {'city': 'Kanpur'}]})` and its output. The output lists 15 documents with fields: roll, name, age, marks, and city. The cities are either 'Lucknow' or 'Kanpur'.

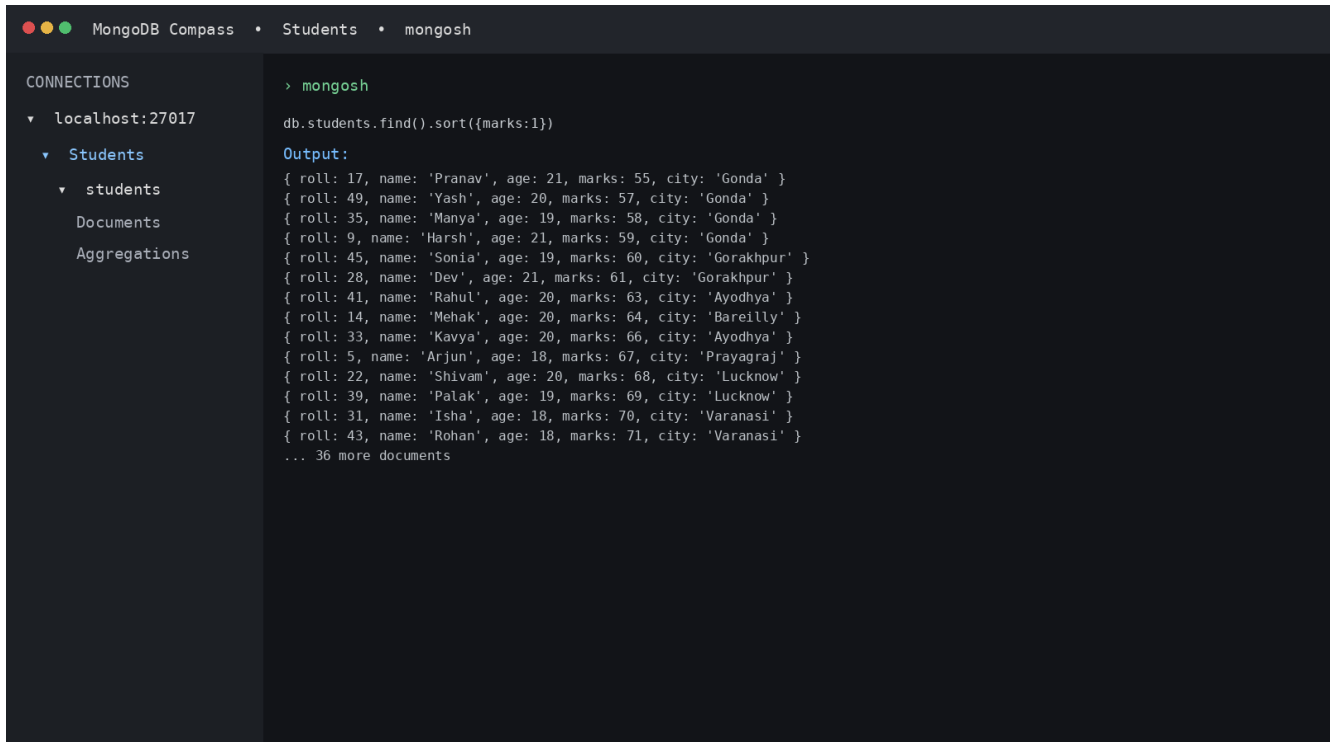
```
> mongosh
db.students.find({'$or': [{'city': 'Lucknow'}, {'city': 'Kanpur'}]})

Output:
{ roll: 1, name: 'Aarav', age: 19, marks: 84, city: 'Lucknow' }
{ roll: 2, name: 'Aditi', age: 18, marks: 91, city: 'Kanpur' }
{ roll: 6, name: 'Bhavya', age: 20, marks: 95, city: 'Lucknow' }
{ roll: 10, name: 'Ishita', age: 19, marks: 90, city: 'Lucknow' }
{ roll: 11, name: 'Kunal', age: 20, marks: 86, city: 'Kanpur' }
{ roll: 12, name: 'Lavanya', age: 18, marks: 78, city: 'Lucknow' }
{ roll: 16, name: 'Pallavi', age: 19, marks: 87, city: 'Lucknow' }
{ roll: 18, name: 'Riya', age: 18, marks: 94, city: 'Kanpur' }
{ roll: 19, name: 'Ritvik', age: 20, marks: 72, city: 'Lucknow' }
{ roll: 22, name: 'Shivam', age: 20, marks: 68, city: 'Lucknow' }
{ roll: 25, name: 'Dhruv Dwivedi', age: 20, marks: 93, city: 'Lucknow' }
{ roll: 26, name: 'Aayush', age: 19, marks: 74, city: 'Kanpur' }
{ roll: 27, name: 'Anika', age: 18, marks: 85, city: 'Lucknow' }
{ roll: 30, name: 'Himanshu', age: 20, marks: 88, city: 'Lucknow' }
... 9 more documents
```

Figure: MongoDB execution screenshot — 14. OR OPERATION — \$or

15A. SORT OPERATION — ASCENDING

The sort operation arranges students by marks from lowest to highest. Ascending order uses 1.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows 'localhost:27017' with a 'Students' database containing a 'students' collection. The main panel shows the command `db.students.find().sort({marks:1})` in the 'mongosh' terminal. The output displays a list of student documents sorted by their 'marks' in ascending order. The documents are as follows:

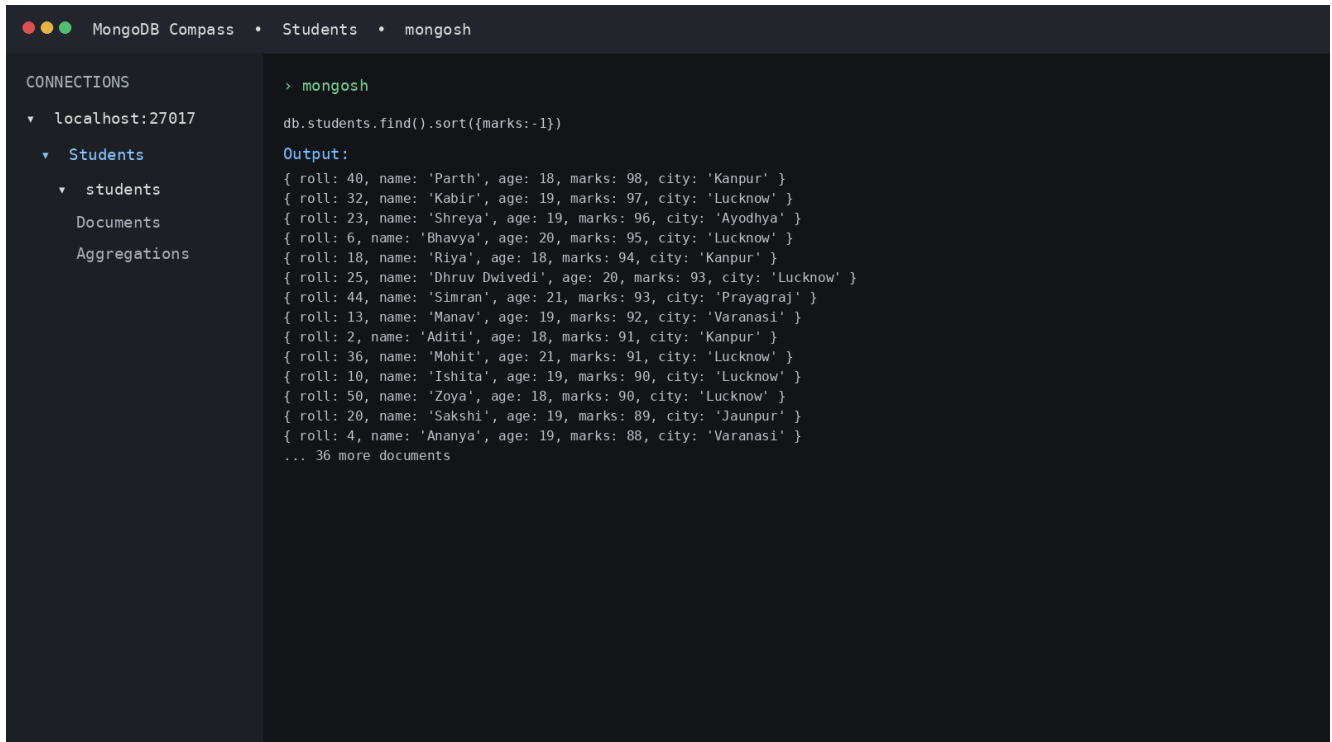
roll	name	age	marks	city
17	Pranav	21	55	Gonda
49	Yash	20	57	Gonda
35	Manya	19	58	Gonda
9	Harsh	21	59	Gonda
45	Sonia	19	60	Gorakhpur
28	Dev	21	61	Gorakhpur
41	Rahul	20	63	Ayodhya
14	Mehak	20	64	Bareilly
33	Kavya	20	66	Ayodhya
5	Arjun	18	67	Prayagraj
22	Shivam	20	68	Lucknow
39	Palak	19	69	Lucknow
31	Isha	18	70	Varanasi
43	Rohan	18	71	Varanasi

... 36 more documents

Figure: MongoDB execution screenshot — 15A. SORT OPERATION — ASCENDING

15B. SORT OPERATION — DESCENDING

The sort operation arranges students by marks from highest to lowest. Descending order uses -1.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017' with a database named 'Students'. The 'students' collection is selected, and the 'Documents' tab is active. The main panel shows the command `db.students.find().sort({marks:-1})` entered in the 'mongosh' shell. The output displays a list of student documents sorted by their 'marks' in descending order. The first document has a mark of 98, and the last visible document has a mark of 88. The output is truncated with '... 36 more documents'.

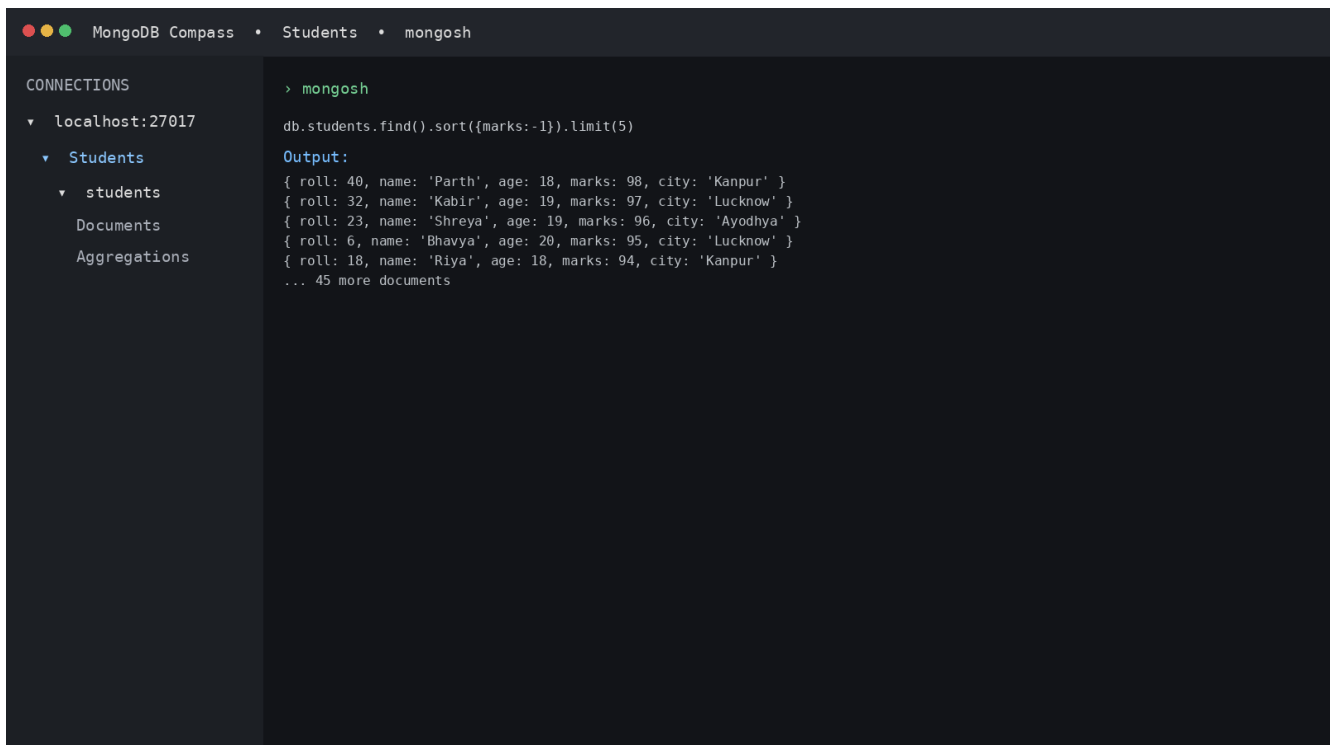
```
> mongosh
db.students.find().sort({marks:-1})

Output:
{ roll: 40, name: 'Parth', age: 18, marks: 98, city: 'Kanpur' }
{ roll: 32, name: 'Kabir', age: 19, marks: 97, city: 'Lucknow' }
{ roll: 23, name: 'Shreya', age: 19, marks: 96, city: 'Ayodhya' }
{ roll: 6, name: 'Bhavya', age: 20, marks: 95, city: 'Lucknow' }
{ roll: 18, name: 'Riya', age: 18, marks: 94, city: 'Kanpur' }
{ roll: 25, name: 'Dhruv Dwivedi', age: 20, marks: 93, city: 'Lucknow' }
{ roll: 44, name: 'Simran', age: 21, marks: 93, city: 'Prayagraj' }
{ roll: 13, name: 'Manav', age: 19, marks: 92, city: 'Varanasi' }
{ roll: 2, name: 'Aditi', age: 18, marks: 91, city: 'Kanpur' }
{ roll: 36, name: 'Mohit', age: 21, marks: 91, city: 'Lucknow' }
{ roll: 10, name: 'Ishita', age: 19, marks: 90, city: 'Lucknow' }
{ roll: 50, name: 'Zoya', age: 18, marks: 90, city: 'Lucknow' }
{ roll: 20, name: 'Sakshi', age: 19, marks: 89, city: 'Jaunpur' }
{ roll: 4, name: 'Ananya', age: 19, marks: 88, city: 'Varanasi' }
... 36 more documents
```

Figure: MongoDB execution screenshot — 15B. SORT OPERATION — DESCENDING

16. LIMIT OPERATION

The limit operation displays only the first five documents returned by the descending marks query.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows 'localhost:27017' with a 'Students' database. The 'students' collection is selected, showing 'Documents' and 'Aggregations'. The main panel shows a query in the 'mongosh' terminal:

```
> mongosh
db.students.find().sort({marks:-1}).limit(5)
```

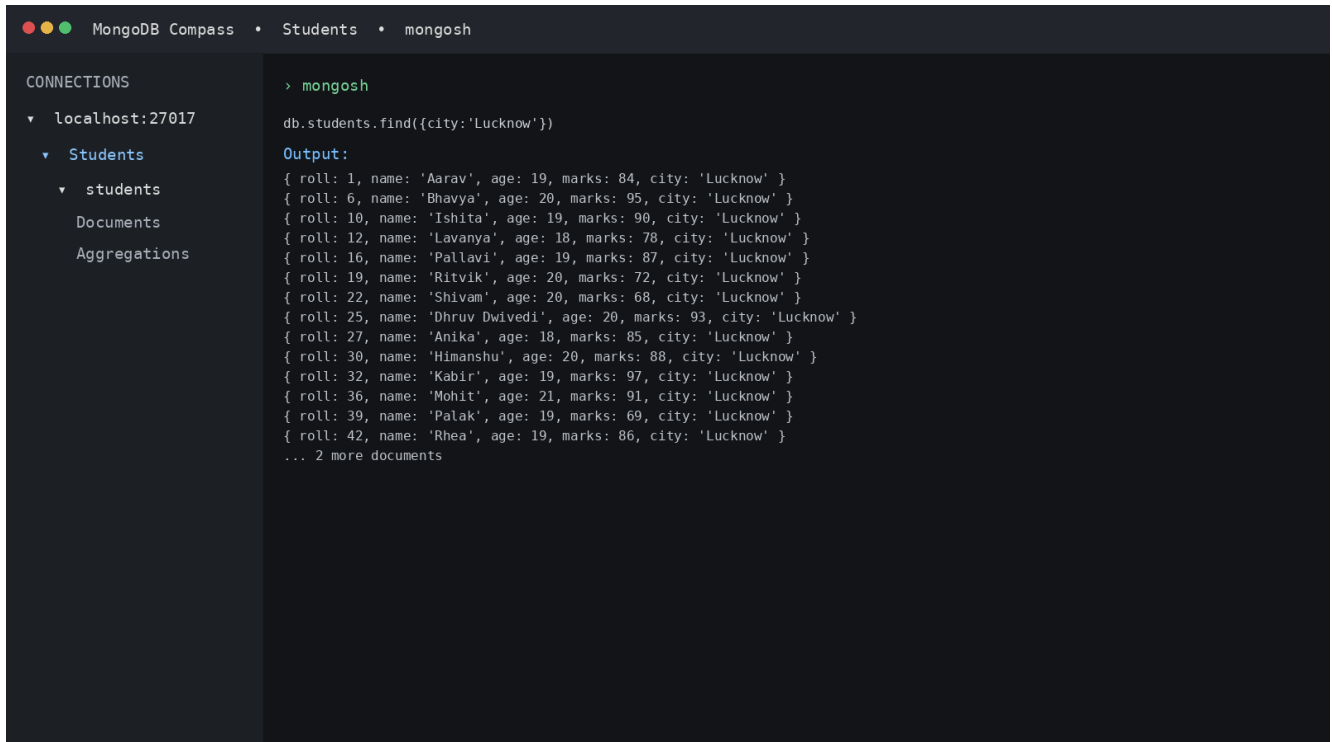
The output of the query is displayed below the terminal:

```
Output:
{ roll: 40, name: 'Parth', age: 18, marks: 98, city: 'Kanpur' }
{ roll: 32, name: 'Kabir', age: 19, marks: 97, city: 'Lucknow' }
{ roll: 23, name: 'Shreya', age: 19, marks: 96, city: 'Ayodhya' }
{ roll: 6, name: 'Bhavya', age: 20, marks: 95, city: 'Lucknow' }
{ roll: 18, name: 'Riya', age: 18, marks: 94, city: 'Kanpur' }
... 45 more documents
```

Figure: MongoDB execution screenshot — 16. LIMIT OPERATION

17. SEARCH BY CITY

The city search finds all students whose city is Lucknow.



The screenshot shows the MongoDB Compass interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017' with a database named 'Students'. The 'students' collection is selected, and the 'Documents' tab is active. The main panel shows the command `db.students.find({city:'Lucknow'})` entered in the 'mongosh' shell. The output displays 15 documents, each with fields for roll number, name, age, marks, and city. All documents have 'city: 'Lucknow''. The output is truncated with '... 2 more documents'.

```
MongoDB Compass • Students • mongosh

CONNECTIONS
  ▼ localhost:27017
    ▼ Students
      ▼ students
        Documents
        Aggregations

> mongosh
db.students.find({city:'Lucknow'})

Output:
{ roll: 1, name: 'Aarav', age: 19, marks: 84, city: 'Lucknow' }
{ roll: 6, name: 'Bhavya', age: 20, marks: 95, city: 'Lucknow' }
{ roll: 10, name: 'Ishita', age: 19, marks: 90, city: 'Lucknow' }
{ roll: 12, name: 'Lavanya', age: 18, marks: 78, city: 'Lucknow' }
{ roll: 16, name: 'Pallavi', age: 19, marks: 87, city: 'Lucknow' }
{ roll: 19, name: 'Ritvik', age: 20, marks: 72, city: 'Lucknow' }
{ roll: 22, name: 'Shivam', age: 20, marks: 68, city: 'Lucknow' }
{ roll: 25, name: 'Dhruv Dwivedi', age: 20, marks: 93, city: 'Lucknow' }
{ roll: 27, name: 'Anika', age: 18, marks: 85, city: 'Lucknow' }
{ roll: 30, name: 'Himanshu', age: 20, marks: 88, city: 'Lucknow' }
{ roll: 32, name: 'Kabir', age: 19, marks: 97, city: 'Lucknow' }
{ roll: 36, name: 'Mohit', age: 21, marks: 91, city: 'Lucknow' }
{ roll: 39, name: 'Palak', age: 19, marks: 69, city: 'Lucknow' }
{ roll: 42, name: 'Rhea', age: 19, marks: 86, city: 'Lucknow' }
... 2 more documents
```

Figure: MongoDB execution screenshot — 17. SEARCH BY CITY

18. COUNT DOCUMENTS

The count operation counts the documents currently present in the collection after the delete operations. The resulting count is 45.

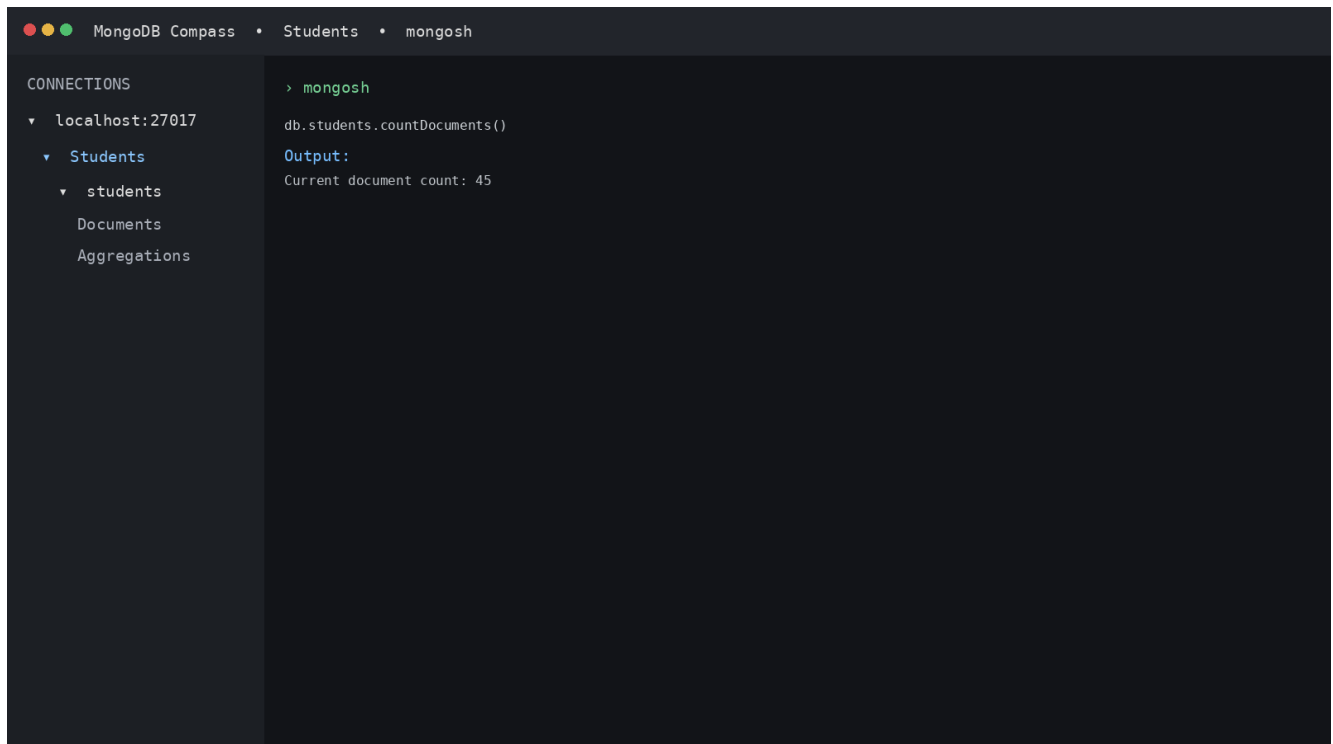


Figure: MongoDB execution screenshot — 18. COUNT DOCUMENTS

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the fresh dataset.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how ascending and descending order work, how comparison operators such as \$eq, \$gt, \$lt, \$gte and \$lte are used, and how logical operators such as \$and and \$or help in filtering data.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.